TRAFFIC LIGHT

Course code: EEC214
Course Name: Microprocessor

Name	ID
احمد بشري احمد	20243166
اسلام السيد غنيمي	20210536
عبد الرحمن ايهاب عيسي	20200580
محمد ثروت محمد	20160772

Abstract

This report presents the development of a microcontroller-based traffic control system that integrates traffic light management, real-time traffic announcements, and environmental monitoring into a single platform. The system is designed to regulate vehicle flow efficiently by controlling the traffic lights (red, yellow, and green) using predefined timing sequences.

To improve usability and communication with road users, a 16×2 LCD module is utilized to display real-time status messages such as “Stop”, “Wait”, and “Go”. Furthermore, the system incorporates a DHT11 temperature and humidity sensor to monitor environmental conditions, with the measured data directly displayed on the LCD.

The proposed system was initially designed and validated through simulation using Proteus Design Suite, followed by hardware implementation. The overall results highlight a reliable, low-cost, and scalable solution that can serve as a foundational model for modern traffic management systems.

Acknowledgement

I would like to express my gratitude to my Doctor Mohamed Torad, whose guidance, and support were crucial in the completion of this work.

I am also grateful to the faculty and staff of HTI for providing an enriching academic environment and access to necessary resources and facilities. Special thanks to all the distinguished doctors for their support and assistance.

My sincere appreciation goes to my team, for their collaboration, discussions, and camaraderie throughout this journey. Your perspectives and support have greatly contributed to this work.

I extend my heartfelt thanks to my family and friends for their unwavering support, patience, and understanding. Your love and encouragement have been my constant source of strength and motivation.

Thank you all for your contributions and support

Chapter 1

Problem Statement

Traditional traffic control systems suffer from several limitations that affect both efficiency and safety:

Operational Limitations:

· Fixed timing signals that do not adapt to real-time traffic conditions

· Lack of traffic announcements or driver guidance

· Inefficient management of traffic flow under varying conditions Safety Issues:

· Absence of clear instructions, which may lead to driver confusion

· Increased risk of accidents, especially at intersections · Lack of real-time information for drivers Lack of Environmental Awareness:

· No monitoring of temperature and humidity conditions

· Environmental factors are not considered in system operation

· No real-time display of surrounding environmental data

To address these challenges, this project proposes a microcontroller-based traffic control system integrated with real-time traffic announcements and

environmental monitoring using a DHT11 temperature and humidity sensor. The system provides a simple, efficient, and informative solution for improved traffic management.

Chapter 2

Components Used

Component	Specification	Quantity	Purpose
Microcontroller	Atmega328p	1	Central system control and traffic logic processing.
LCD Display	16*2 LCD	1	Real-time status interface and environment data display
I2C Interface Module	PCF8574	1	Enabling I2C communication to reduce wiring for the LCD.
Environment Sensor	DHT11	1	Measuring ambient temperature and humidity levels.

7-Segment Display	Common Cathode	2	Displaying the visual countdown for traffic signal phases.
7-Segment Decoder	74LS48BCD	2	Decoding BCD signals to drive the 7segment display.

1. AVR_USPASP: Enables serial communication.

figure . 1 AVR-USPASP

2. Atmega32 microcontroller: Brain of the system, controls all operations.

figure . 2 Atmega32 microcontroller

3. LCD

figure 3 lcd

4. I2C Interface Module (PCF8574) Function: An 8-bit I/O expander that converts parallel data from the LCD into serial data using the I2C bus. Communication: Uses only two pins (SDA for data and SCL for clock) to communicate with the ATmega328P. Efficiency: Saves up to 6 microcontroller pins, allowing them to be used for other components like sensors and LEDs. Addressing: Features adjustable hardware address pins (A0, A1, A2) to allow multiple devices on the same bus. Integration: Equipped with a built-in potentiometer for easy adjustment of the LCD's backlight and contrast. Purpose: Simplifies hardware wiring and optimizes the microcontroller’s resource management for the overall system.

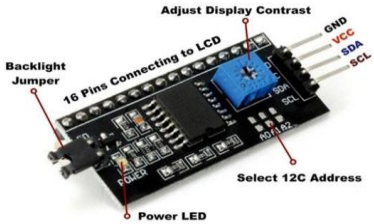


figure 4 Interface Module

5.Wires: connect components together.

figure 5 Interface Module

6.DHT11 Environment Sensor

Display: Two lines with 16 characters each using 5×8 pixel matrix format. **Interface**: 4-bit mode requiring only 6 control lines for efficient pin usage. **Controller**: HD44780 compatible with built-in character generator ROM. **Backlight**: LED illumination with adjustable brightness through current limiting.

Control: Cursor positioning, display clearing, and custom character creation capabilities.

figure 7 DHT

7.7-Segment Display

Display: Dual-digit, common-cathode configuration used to display numeric countdowns.

Structure: Consists of 7 individual LEDs (segments) arranged in an "8" shape plus a decimal point.

Interface: Driven via the 74LS48 decoder to convert BCD signals into 7-segment patterns.

Visibility: High-brightness output ensuring clear readability of the remaining time for traffic phases.

Control: Multiplexed or direct drive logic to minimize microcontroller I/O pin consumption.

Purpose: Acts as a visual countdown timer to enhance driver awareness and intersection safety.

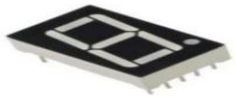


figure 8 7- Segment

Chapter 3

3.1 The Pin Configurations of The controller:

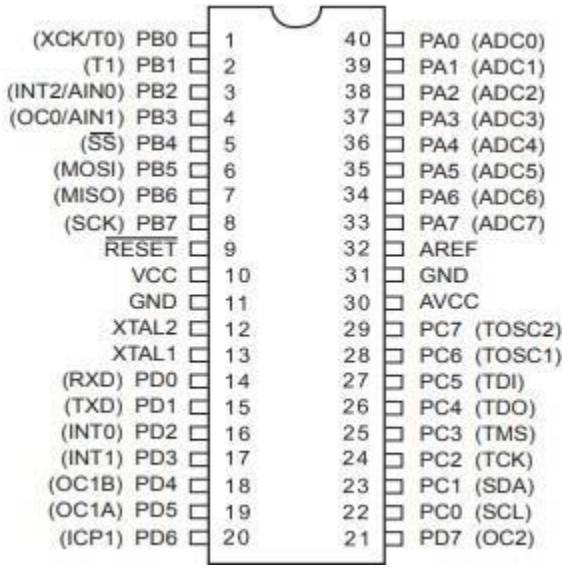


figure 9 pin configuration of the controller

1.2.1 ATmega32

Architecture: 8-bit RISC processor with Harvard architecture for enhanced performance.

Memory: 32KB Flash, 2KB SRAM, 1KB EEPROM for program and data storage.

I/O Ports: Four 8-bit bidirectional ports (32 pins) with programmable pull-up resistors.

Clock: External 8MHz crystal with internal oscillator options and power management.

Features: PWM, USART, analog comparator, and watchdog timer for system

Reliability .

Chapter 4

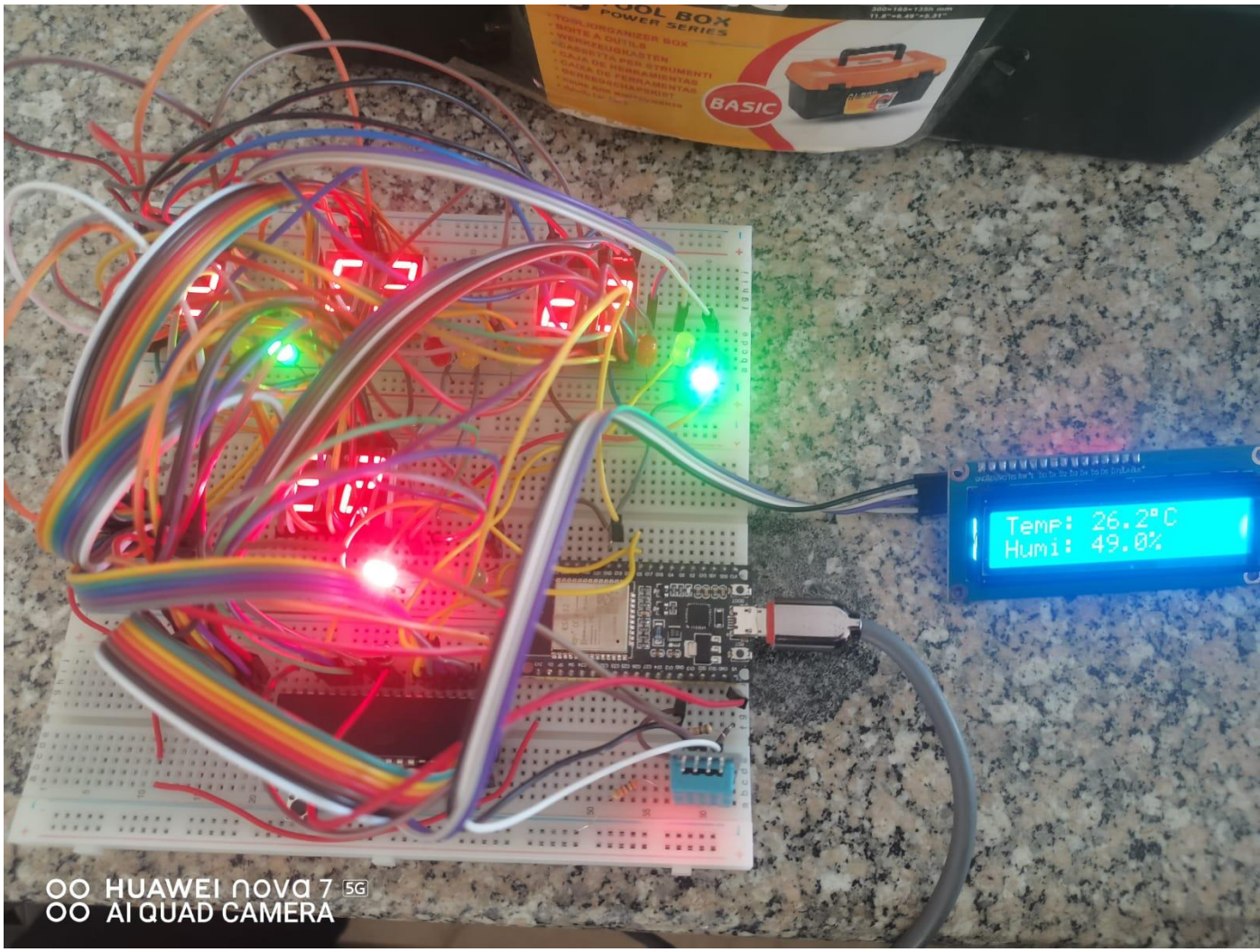
Implementation

4.1 Applications used:

Arduino IDE : for burning the hex file

4.1 Hardware

A real picture for the project



C_code.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_Sensor.h>
#include <DHT.h>

#define DHT11_PIN   11
#define DHTTYPE     DHT11

LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHT11_PIN, DHTTYPE);

float temperature = 0;
float humidity    = 0;
unsigned long lastDHTRead = 0;
bool lcdOK = false;
bool fastMode = false;
bool lastButtonState = true;

const byte digits[10] = {
   0b11000000, 0b11111001, 0b10100100,
           0b10110000,
   0b10011001, 0b10010010, 0b10000010,
           0b11111000,
   0b10000000, 0b10010000
};

#define RED_BIT     0
#define GREEN_BIT   1
#define YELLOW_BIT  2
#define SEC_LED_BIT 4
#define BUTTON_BIT  2

void showNumber(int num) {
   PORTB = digits[num % 10];
   PORTA = digits[num / 10];
}

void allLedsOff() {
   PORTD &= ~(0b00010111);
}

void checkButton() {
```

```
  bool currentState = (PINC >> BUTTON_BIT) & 1;
  if (currentState == false && lastButtonState == true)
          {
    fastMode = !fastMode;
    delay(200);
  }
  lastButtonState = currentState;
}

int getDelay() {
  return fastMode ? 100 : 500;
}

void updateLCD() {
  if (!lcdOK) return;
  if (millis() - lastDHTRead < 2000) return;
  lastDHTRead = millis();

  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    lcd.setCursor(0, 0);
    lcd.print("Sensor Failed   ");
    lcd.setCursor(0, 1);
    lcd.print("                ");
    return;
  }

  temperature = t;
  humidity    = h;

  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C   ");

  lcd.setCursor(0, 1);
  lcd.print("Humi: ");
  lcd.print(humidity, 1);
  lcd.print("%   ");
}

void setup() {
  DDRA  = 0xFF;
  DDRB  = 0xFF;
```

```
  DDRD |= 0b00010111;


  DDRC  &= ~(1 << BUTTON_BIT);
  DDRC  &= ~(1 << 5);
  PORTC |=  (1 << BUTTON_BIT);

  allLedsOff();

  Wire.begin();
  Wire.beginTransmission(0x27);
  if (Wire.endTransmission() == 0) {
    lcdOK = true;
    lcd.begin();
    lcd.backlight();
  }

  dht.begin();
  delay(2000);
}

void loop() {


  allLedsOff();
  PORTD |= (1 << GREEN_BIT);

  for (int i = 40; i >= 0; i--) {
    checkButton();
    if (i <= 4 && i > 0) {
      PORTD &= ~(1 << GREEN_BIT);
      PORTD |=  (1 << YELLOW_BIT);
    } else if (i == 0) {
      PORTD &= ~(1 << GREEN_BIT);
      PORTD &= ~(1 << YELLOW_BIT);
    } else {
      PORTD &= ~(1 << YELLOW_BIT);
      PORTD |=  (1 << GREEN_BIT);
    }
    showNumber(i);
    updateLCD();
    delay(getDelay());
  }


  allLedsOff();
  PORTD |= (1 << RED_BIT);
```

```
    for (int i = 40; i >= 0; i--) {
        checkButton();
        if (i <= 4 && i > 0) {
            PORTD &= ~(1 << RED_BIT);
            PORTD &= ~(1 << YELLOW_BIT);
            PORTD |=  (1 << SEC_LED_BIT);
        } else if (i == 0) {
            PORTD &= ~(1 << RED_BIT);
            PORTD &= ~(1 << YELLOW_BIT);
            PORTD &= ~(1 << SEC_LED_BIT);
        } else {
            PORTD &= ~(1 << YELLOW_BIT);
            PORTD &= ~(1 << SEC_LED_BIT);
            PORTD |=  (1 << RED_BIT);
        }
        showNumber(i);
        updateLCD();
        delay(getDelay());
    }
}
```

Chapter 5

Results traffic control system with traffic announcements was successfully implemented with the following outcomes:

Functional Requirements Met:

o Proper sequencing of traffic lights (Red, Yellow, Green) achieved

o Smooth transition between signals based on predefined timing

o Real-time status displayed clearly on LCD

o Traffic announcements synchronized with signal changes

o System responds correctly to user input (if keypad used)

Reliability:

o Accurate timing performance during multiple test cycles

o No system crashes or unexpected behavior observed

o Stable operation over extended testing periods

o Consistent synchronization between lights and announcements

User Experience:

o Clear and readable information displayed on LCD

o Fast and responsive system interaction

o Audio announcements enhance driver awareness

o Simple and intuitive system operatio

References

1. Muhammad Ali Mazidi, The AVR Microcontroller and Embedded Systems.
2. Frank Vahid, Embedded Systems Design: A Unified Hardware/Software Introduction.
3. Atmel Corporation, ATmega32 Datasheet.
4. Microchip Technology, AVR Microcontroller Documentation.
5. Proteus Design Suite Documentation.
6. AVR Studio User Guide.
7. CodeVisionAVR Compiler Manual.
8. C Programming Language Documentation.
9. Traffic Control Systems, Fundamentals and Applications.
10. Embedded Systems Design Concepts and Practices.
11. Microcontroller-Based Systems, Principles and Applications.
12. Digital Control Systems, Theory and Implementation.